data/02/bin/factory_bot.rb
```
ENV["RAILS_ENV"] ||= "test"
require File.expand_path("../../config/environment", __FILE__)
begin
  DatabaseCleaner.start
  FactoryBot.lint(traits: true)
ensure
  DatabaseCleaner.clean
end
```

For this to work, you need to add gem "database_cleaner" in the :test group of the Gemfile. The first two lines of this code are exactly what RSpec does to force execution in the test environment: set the environment variable and load the config/environment.rb file. The FactoryBot.lint call is set with traits: true so that it will verify all traits—in isolation, it won't build up a combinatorial explosion of combined traits. Also, surround it with Database Cleaner calls to make sure anything it saves to the database goes away.

If you run this from the command line with

```
$ ruby bin/factory_bot.rb
```

you get three validation fails, which is impressive given that you have only two factories:

```
* task - Validation failed: Project must exist (ActiveRecord::RecordInvalid)
* trivial - undefined method `due_date=' for #<Task:<ID>> (NoMethodError)
* panic - undefined method `due_date=' for #<Task:<ID>> (NoMethodError)
```

The first failure in task is because in Rails these days, the belongs_to association in Task sets up a validation, which makes sense in production but somewhat goes against the association usage guidelines I listed earlier. You can fix this by adding a project to the Task factory—as long as Tasks are created with build_stubbed it won't really be an issue, but when you use create it's the first step toward slower specs.

The other two fails are embarrassing. due_date isn't an attribute of Task at the moment; it's an attribute of Project, but that's the kind of thing we run linters to find out, I suppose. Fix it by either moving the traits to Project or by adding the attribute to Task.

Preventing Factory Abuse

The initial temptation when using factories is to continue to build large trees of objects—this is particularly true if you're converting a project that was using fixtures. The best way to use factories is to create only the smallest amount of data needed to expose the issue in each test. This practice speeds